Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	Yes
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
AI-Powered-Debt-Relief-Financial-Recovery-Platform/
|
|── backend/
|       |── app.py
|       |── database.py
|       |── models.py
|       |── schemas.py
|       |── auth.py
|       |── ai_service.py
|       |── financial.py
|       |── requirements.txt
|       └── routers/
|               |── users.py
|               |── loans.py
|               |── settlement.py
|               └── dashboard.py

|── frontend/
|       |── src/
|       |── public/
|       |── package.json
|       └── vite.config.js

|── README.md
└── .gitignore
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not yet deployed — runs locally at http://localhost:8501 (frontend) with the API at http://localhost:8000.
Login Credentials (Demo)	No pre-seeded demo account exists; the evaluator should register a new account via the app's Register tab, then log in with those credentials.
Platform / Hosting Provider	Not yet hosted. Docker with Render (or AWS EC2 / Elastic Beanstalk) is the proposed hosting approach documented in the Technology Stack section — not yet implemented.
Repository Link	https://github.com/Chithra-1611/AI-Powered-Debt-Relief-Financial-Recovery-Platform
Demo Video Link	https://drive.google.com/file/d/1xVMljKWnVqeZTEXmsQjVZbiJy2IoB-f1/view?usp=drivesdk

SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:

- Python 3.11+
- Node.js 18+ and npm 9+
- Git
- SQLite (comes with Python)
- Google Gemini API Key

Step 1: Clone the repository: git clone https://github.com/Chithra-1611/AI-Powered-Debt-Relief-Financial-Recovery-Platform.git

Step 2: Navigate to the project directory: cd AI-Powered-Debt-Relief-Financial-Recovery-Platform

Step 3: Setup and run the backend (FastAPI):

- cd backend
- Create a virtual environment: python -m venv venv
- Activate virtual environment: venv\Scripts\activate (Windows)
  source venv/bin/activate (Mac/Linux)
- Install Python dependencies: pip install -r requirements.txt
- Run the FastAPI server: uvicorn app:app --reload --host 0.0.0.0 --port 8000

Step 4: Setup and run the frontend (React):

- Open a new terminal and navigate to the frontend folder: cd frontend
- Install Node dependencies: npm install
- Start the React development server: npm run dev

Step 5: Open your browser and visit: http://localhost:5173

Step 6: Register a new account from the Register tab.

Step 7: Login with the created credentials.

Step 8: Explore dashboard, add loans, analyze financial health, predict settlements and generate AI negotiation letters.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	AI responses depend on Google Gemini API availability	Works when API is active and internet is available
2	Settlement predictions are based on historical patterns and may vary	Use as a financial guidance tool, not as a guaranteed result
3	Currently runs locally; not deployed publicly	Deployment on Render (or similar) is in progress